BUREAU D'ETUDE

Dans le cadre du cursus

EEEA

Architecture et programmation en assembleur
U.E. N5EE02B

BE : Robot

Décembre 2025
Groupe F

Étudiants :
Karam Abdelkerim
Sophie Loupias



Sommaire

I) Introduction **2**

II) Principe du décodage - fractionnement de l'espace adressable **3**

III) Tests des différents périphériques **5**
1) Test du bloc de génération des signaux de sélection 5
2) Périphérique d'afficheur 7 Segments 6
3) Périphérique d'afficheur à LEDs 7
4) Périphérique de gestion de Switches 7
5) Périphérique de gestion du joystick et des capteurs 7
6) Périphérique de commande des moteurs 10
7) Périphérique de mesure distance 11
8) Périphérique de gestion du capteur de ligne 12

IV) Présentation des différents algorithmes de contrôle **12**
1) Mode 0 : Joy v0 12
2) Mode 1 : Joy v1 13
3) Mode 2 : Autonome 13
4) Mode 3 : Esclave 14

V) Conclusion **15**

I) Introduction

Au cours de ce BE nous allons approfondir et consolider nos acquis en architecture et programmation d'un assembleur. Le travail se divise en 2 parties. En premier lieu, une partie hardware qui permettra d'appréhender les différentes fonctionnalités du robot, étudié le microprocesseur et le décodage d'adresse associé, et le développement d'un code C. Cette approche théorique sera ensuite validée par des tests expérimentaux qui assureront le fonctionnement de chaque périphérique et mettront en avance les échanges dynamiques. En second lieu, une partie software et l'expérimentation avec diverses stratégies de déplacement. La bibliothèque de fonctions C nous permettra d'accéder aux périphériques souhaités pour réaliser 2 types de déplacement, un à l'aide de joystick, l'autre en déplacement autonome et suivi de ligne.

II) Principe du décodage - fractionnement de l'espace adressable

On travaille ici avec différentes fonctionnalités du robot BoB comprenant une unité centrale UC, son micro-processeur, le robot et ses capteurs. Afin de comprendre kommment fonctionne notre robot, on s'intéresse à l'architecture de l'Unité de Commande. L'UC est une carte de commande DE1-SoC qui contient un FPGA, des afficheurs 7 segments, des switchs et des leds.

Nous allons dans un premier temps configurer un processeur de type NIOS qui nous permet d'accéder à un bus de données de 32 bits. Pour simplifier l'étude des périphériques d'entrées sorties en représentant le processeur simplement avec son accès aux périphériques externes. On dispose d'un espace adressable de 64 bits pour gérer les périphériques d'entrées et sorties. Puis cet espace est fractionné en 8 périphériques de 8 octets.

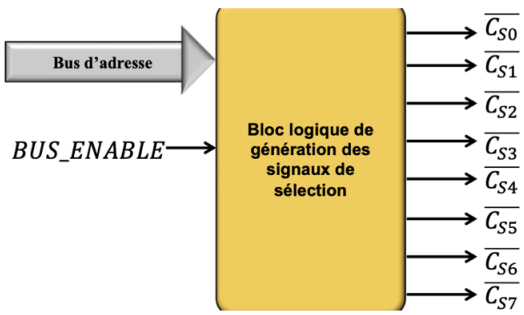
$\overline{C_{S0}}$ - 4*16bits	Périphérique d'affichage pour pilotage des afficheurs 7 segments
$\overline{C_{S1}}$ - 4*16bits	Périphérique d'affichage pour pilotage des leds
$\overline{C_{S2}}$ - 4*16bits	Périphérique d'affichage pour pilotage des moteurs
$\overline{C_{S3}}$ - 4*16bits	Périphérique de gestion des capteurs et du joystick
$\overline{C_{S4}}$ - 4*16bits	Périphérique de contrôle des switches
$\overline{C_{S5}}$ - 4*16bits	Périphérique de mesure distance obstacle centre
$\overline{C_{S6}}$ - 4*16bits	Périphérique de mesure distance obstacle gauche
$\overline{C_{S7}}$ - 4*16bits	Périphérique de mesure distance obstacle droit

Espace adressable

Chaque périphérique est associé à une plage d'adresses mémoire définie au sein de l'espace adressable. Le processeur communique avec ces périphériques par des opérations de lecture et d'écriture mémoire. Il est donc nécessaire de mettre en œuvre un décodage d'adresses afin d'identifier correctement le périphérique ciblé lors d'un accès. Un décodage serré est utilisé afin d'éviter les pertes dans les cases mémoire.

Il s'agit de générer des signaux de sélection, actifs à l'état bas, qui permettont l'activation du périphérique situé dans la zone mémoire concernée.

Ces signaux de sélection seront générés par un bloc de logique combinatoire qui recevra le bus d'adresse et un signal BUS_ENABLE, actif à l'état haut quand les adresses du bus peuvent être exploitées. Quand le signal BUS_ENABLE est à l'état bas, tous les signaux de sélection doivent être inactifs (donc à l'état haut).



On dispose du tableau d'adressage relatif à chaque zone ce qui nous permet d'obtenir les relations suivantes :

$$\overline{C_{ZONE0}} = \overline{A_5} \cdot \overline{A_4} \cdot \overline{A_3}$$
$$\overline{C_{ZONE1}} = \overline{A_5} \cdot \overline{A_4} \cdot A_3$$
$$\overline{C_{ZONE2}} = \overline{A_5} \cdot A_4 \cdot \overline{A_3}$$
$$\overline{C_{ZONE3}} = \overline{A_5} \cdot A_4 \cdot A_3$$
$$\overline{C_{ZONE4}} = A_5 \cdot \overline{A_4} \cdot \overline{A_3}$$
$$\overline{C_{ZONE5}} = A_5 \cdot \overline{A_4} \cdot A_3$$
$$\overline{C_{ZONE6}} = A_5 \cdot A_4 \cdot \overline{A_3}$$
$$\overline{C_{ZONE7}} = A_5 \cdot A_4 \cdot A_3$$

En prenant en compte le signal BUS_ENABLE, on obtient l'équation définitive des signaux de sélection :

$$\overline{CS_x} = \overline{BUS_ENABLE} + \overline{C_{ZONE_x}}$$

BUS_ENABLE	$\overline{\text{CZONE}_x}$	$\overline{\text{Cs}_x}$
0	0	1
0	1	1
1	0	0
1	1	1

Enfin, on implémente ces équations dans Quartus puis on télécharges le fichier dans le FPGA afin de visualiser l'activation des signaux $\overline{CS_x}$.

III) Tests des différents périphériques

1) Test du bloc de génération des signaux de sélection

Pour cette série de tests on va utiliser le logiciel NIOS II. Dans un premier temps, on écrit un programme qui nous permet d'accéder aux différentes zones mémoires. On déclare l'adresse des 8 zones mémoire dans notre code :

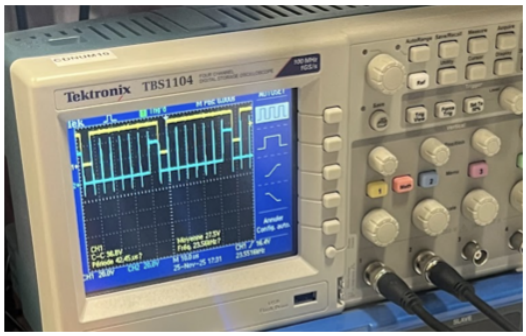
```
#define CS_0 ((volatile short* (TO_EXTERNAL_BUS_BRIDGE_0_BASE))
#define CS_1 ((volatile short*) (TO_EXTERNAL_BUS_BRIDGE_0_BASE+8))
#define CS_2 ((volatile short*) (TO_EXTERNAL_BUS_BRIDGE_0_BASE+16))
#define CS_3 ((volatile short*) (TO_EXTERNAL_BUS_BRIDGE_0_BASE+24))
#define CS_4 ((volatile short*) (TO_EXTERNAL_BUS_BRIDGE_0_BASE+32))
#define CS_5 ((volatile short*) (TO_EXTERNAL_BUS_BRIDGE_0_BASE+40))
#define CS_6 ((volatile short*) (TO_EXTERNAL_BUS_BRIDGE_0_BASE+48))
#define CS_7 ((volatile short*) (TO_EXTERNAL_BUS_BRIDGE_0_BASE+56))
```

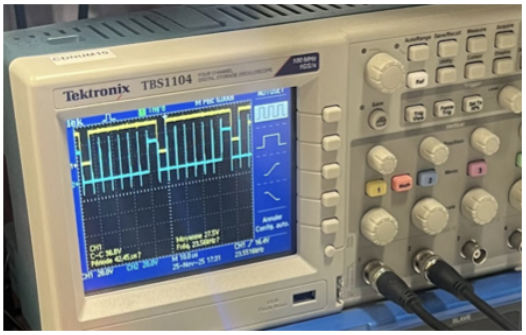
```
short var;

while (1) {
        *CS_0 = 85; // Ecriture 2 premiers octets ZONE0
        *CS_1 = 85; // Ecriture 2 premiers octets ZONE1
        *CS_2 = 85; // Ecriture 2 premiers octets ZONE2
        *CS_3 = 85; // Ecriture 2 premiers octets ZONE3
        *CS_4 = 85; // Ecriture 2 premiers octets ZONE4
        *CS_5 = 85; // Ecriture 2 premiers octets ZONE5
        *CS_6 = 85; // Ecriture 2 premiers octets ZONE6
        var=*CS_7 ; // Lecture  2 premiers octets ZONE7
}
```

A l'aide d'une boucle infinie d'écriture sur les différentes zones on visualise les 8 signaux d'activation ainsi que les signaux R/W et *BUS_ENABLE* à l'oscilloscope en prenant le signal CS0 comme référence.

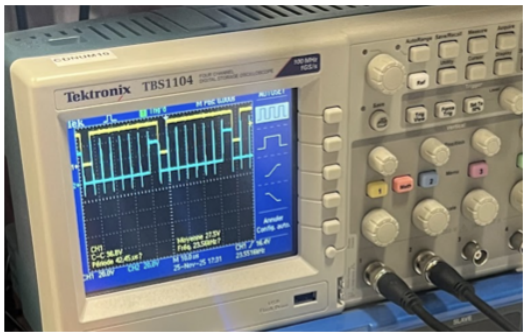
On observe ici le signal R/W (bleu). On constate notamment un pic juste avant le pic du signal CS0 (jaune) ce qui veut dire il ya une mise en place du type d'accès avant la validation du périphérique lors du décodage d'adresse.





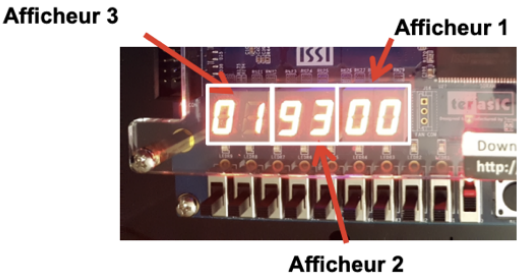
On observe le signal *BUS_ENABLE* (bleu). Celui-ci est toujours à l'état haut et on voit bien ce phénomène ici par la répétition de pics resserrés.

On observe le signal R/W (bleu). On constate notamment un pic juste avant le pic du signal CS0 (jaune)indiquant qu'il ya une mise en place du type d'accès avant la validation du périphérique lors du décodage d'adresse. L'échelle de notre oscilloscope indique qu'un carreau correspond à 10 μs. Alors d'après la visualisation du signal R/W on voit bien qu'il faut environ 5 μs pour que l'écriture ou la lecture s'exécute.



2) Périphérique d'afficheur 7 Segments

Le but de ce module est de contrôler les trois afficheurs de la carte FPGA. Ils seront nommés comme ceci :



Ces trois afficheurs permettent d'afficher un nombre entre 0 et 99. Au-delà de ce seuil, l'afficheur affiche EE signifiant qu'il y a une erreur.

On teste le bon fonctionnement de ce périphérique en implémentant le programme sur NIOS II.

```
int main()
{
    // Ecrire Coucou sur la console
    alt_putstr("Hello from Nios II!\n");

    // Envoyer des données sur les 3 afficheurs
    *AFFICHEUR_1 = 1;
    *AFFICHEUR_2 = 100; // afficahge code erreur car >99
    *AFFICHEUR_3 = 93;


    /* Event loop never exits. */
    while (1) {
    };

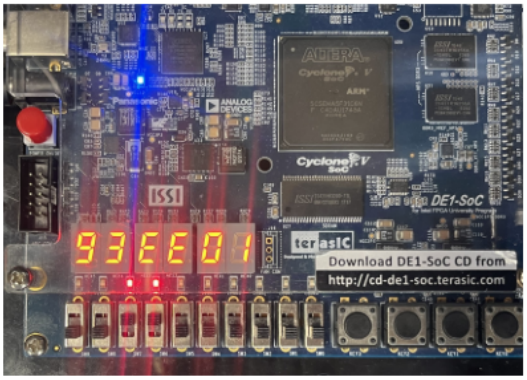
    return 0;
}
```

Une fois le fonctionnement validé, on réalise 3 sous programmes $PILOTAGE_AFFICHEUR\ 1, 2, 3$ avec pour argument la valeur à afficher.

```
21
22 void PILOTAGE_AFFICHEUR_1(short valeur)
23 {
24     *AFFICHEUR_1 = valeur;
25 }
26
27 void PILOTAGE_AFFICHEUR_2(short valeur)
28 {
29     *AFFICHEUR_2 = valeur;
30 }
31
32 void PILOTAGE_AFFICHEUR_3(short valeur)
33 {
34     *AFFICHEUR_3 = valeur;
35 }
```

```
38
39 int main(void)
40 {
41     short var;
42
43
44     alt_putstr("Hello from Nios II!\n");
45
46
47     PILOTAGE_AFFICHEUR_1(1);
48     PILOTAGE_AFFICHEUR_2(100);
49     PILOTAGE_AFFICHEUR_3(93);
50     *LED_ROUGES = 0xAA;
```

Enfin, on observe l'affichage des valeurs sur les 3 afficheurs : On note le EE, signifiant que la valeur demandée est supérieure à 99.



3) Périphérique d'afficheur à LEDs

On souhaite dans cette partie contrôler l'affichage des 10 LEDs rouges ($LEDR_0, \ldots, LEDR_9$) situés sur la carte DE1-SoC. On accède aux LEDR par les adresses 0x08 et 0x09. Pour l'ensemble des 10 LEDs on associe un registre codé sur 16 bits qui contient les nombres de 0 à 1023. Ensuite, on veut écrire un programme pour tester le fonctionnement du périphérique.

```
int main()
{
    // Ecrire Coucou sur la console
    alt_putstr("Hello from Nios II!\n");

    // Envoyer la donnée 10101010 sur les LEDS rouges
    *LED_ROUGES = 0xAA;

    /* Event loop never exits. */
    while (1) {
    };

    return 0;
}
```

4) Périphérique de gestion de Switches

De même que pour les LEDs on accède en lecture au statut des 10 switches ($SW_0, \ldots, SW_9$). On associe au périphérique un registre de 16 bits où seulement les bits de poids faible 10 à 0 sont utilisés. Chaque bit permet d'accéder à la lecture de l'état d'un switch. On teste ensuite le bon fonctionnement de ce périphérique en testant le programme ci-contre :

```
int main()
{
    // Ecrire Coucou sur la console
    alt_putstr("Hello from Nios II!\n");

    /* Event loop never exits. */
    while (1) {
        *LED_ROUGES = *SWITCHES  ;
    };

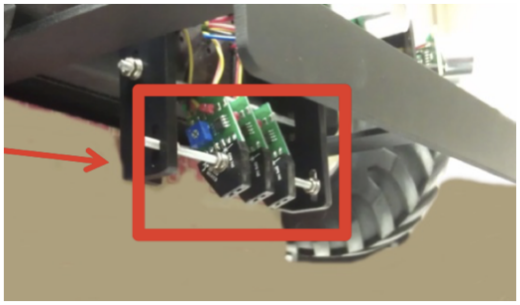
    return 0;
}
```

Une fois validé, on écrit le sous-programme de lecture des switchs :

```
1 int LECTURE_SWs(void)
2 {
3     return (*SWITCHES) & 0x03FF;
4 }
```

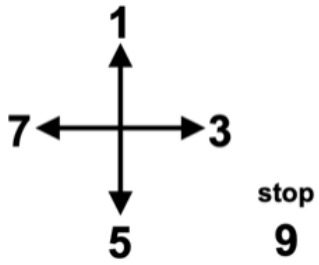
5) Périphérique de gestion du joystick et des capteurs

Ce périphérique permet d'accéder à l'état des 3 capteurs présents sur le robot et de récupérer au sein du processeur l'état du joystick.



Le périphérique contient 2 registres de 16 bits accessibles en lecture seulement :`REGISTRE_CAPTEURS_ROBOT` et `REGISTRE_JOYSTICK`.

Le premier registre se trouve aux adresses $0x10$ à $0x11$, et utilise seulement les 3 bits de poids faible pour donner accès à l'état du capteur droit, gauche et centre. Le deuxième registre se trouve aux adresses $0x12$ à $0x13$, et utilise les 5 bits de poids faibles pour donner l'état du bouton et des positions droite, gauche, haut, bas. Ensuite on teste le bon fonctionnement de ce périphérique en réalisant un programme qui permet de lire l'état du Joystick et afficher la valeur sur l'afficheur 1.

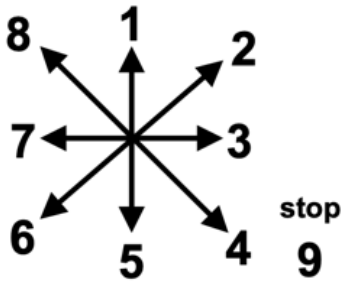


On a ici une photo qui décrit les valeurs qui doivent être lues sur l'afficheur en fonction des positions du Joystick.

On implémente aussi une ligne de code pour afficher la valeur zéro lorsqu'aucune action n'est prise.

Une fois le fonctionnement vérifié, on réécrit le code en ajoutant les positions diagonales du joystick et en imposant une priorité à l'arrêt.

La valeur devant être lue sur l'afficheur est explicitée par la figure ci-contre :



On écrit le code suivant :

```
19
20  void main() {
21      short capteurs;
22      short vTestJoy;
23      init_systeme();
24
25      while(1) {
26          vTestJoy = *CAPTEURS_JOYSTICK;
27          capteurs = *CAPTEURS_ROBOT;
28          if (TEST_JOY_HAUT(vTestJoy) && TEST_JOY_DROITE(vTestJoy)) {
29              *COMMANDE_MOTEUR_DROIT = STOP;
30              *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
31          }
32          else if (TEST_JOY_HAUT(vTestJoy)) {
33              *COMMANDE_MOTEUR_DROIT = STOP + MARCHE_AV;
34              *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
35          }
36          else if (TEST_JOY_BAS(vTestJoy)) {
37              *COMMANDE_MOTEUR_DROIT = STOP + MARCHE_AR;
38              *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;
39          }
40          else {
41              *COMMANDE_MOTEUR_DROIT = STOP;
42              *COMMANDE_MOTEUR_GAUCHE = STOP;
43          }
44          if (TEST_CAPTEUR_CENTRE(capteurs) == 0) {
45              *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV/2;
46              *COMMANDE_MOTEUR_DROIT = STOP + MARCHE_AV/2;
47          }
48          else if (TEST_CAPTEUR_GAUCHE(capteurs) == 0) {
49              *COMMANDE_MOTEUR_GAUCHE = STOP;
50              *COMMANDE_MOTEUR_DROIT = STOP + MARCHE_AV/2;
51          }
52
53          switch (vTestJoy) {
54              case 16: PILOTAGE_AFFICHEUR_1(1); break; // Avant
55              case 8:  PILOTAGE_AFFICHEUR_1(5); break; // Arrière
56              case 2:  PILOTAGE_AFFICHEUR_1(3); break; // Droite
57              case 4:  PILOTAGE_AFFICHEUR_1(7); break; // Gauche
58              case 1:  PILOTAGE_AFFICHEUR_1(9); break; // Arrêt
59              default:
60
61                  if (vTestJoy % 2 != 0) PILOTAGE_AFFICHEUR_1(9);
62                  else PILOTAGE_AFFICHEUR_1(0);
63                  break;
64          }
65          PILOTAGE_AFFICHEUR_2(*ECHO_CENTRE);
66      }
67  }
```

Programme de gestion du joystick

Enfin, on souhaite créer 5 sous-programmes qui à partir de la valeur lue sur le registre Joystick renvoie 1 ou 0 en fonction de la position du joystick (1 si direction activée, 0 sinon)

Pour cela on va utiliser le principe du masquage qui permet d'isoler le bit qu'on souhaite lire. Chaque bit (0 à 5) correspond à une position du Joystick comme nous montre la figure ci-contre :

REGISTRE_JOYSTICK (16 bits)

Bit	Signal
DATA[0]	Etat du bouton
DATA[1]	Direction droite
DATA[2]	Direction gauche
DATA[3]	Direction arrière
DATA[4]	Direction avant
DATA[15..5]	0 (Non significatif)

Par exemple, si on souhaite tester la position droite, on utilise le masque 0x0002, puis on décale le bit d'une position afin de renvoyer 1 si la position droite est bien activée.
On écrit donc les sous-programmes ci-dessous :

```
short TEST_BOUTON(short v) { return (v & 0x0001); }
short TEST_DROITE(short v) { return (v & 0x0002) >> 1; }
short TEST_GAUCHE(short v) { return (v & 0x0004) >> 2; }
short TEST_BAS(short v)    { return (v & 0x0008) >> 3; }
short TEST_HAUT(short v)   { return (v & 0x0010) >> 4; }
```

On modifie le code précédent et on écrit :

```
void main() {
    short capteurs, vTestJoy;
    init_systeme();

    while(1) {
        vTestJoy = *CAPTEURS_JOYSTICK;
        capteurs = *CAPTEURS_ROBOT;

        if (TEST_JOY_HAUT(vTestJoy) && TEST_JOY_GAUCHE(vTestJoy)) {
            *COMMANDE_MOTEUR_DROIT = STOP + MARCHE_AV;
            *COMMANDE_MOTEUR_GAUCHE = STOP;
        }
        else if (TEST_JOY_HAUT(vTestJoy) && TEST_JOY_DROITE(vTestJoy)) {
            *COMMANDE_MOTEUR_DROIT = STOP;
            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
        }
        else if (TEST_JOY_BAS(vTestJoy) && TEST_JOY_GAUCHE(vTestJoy)) {
            *COMMANDE_MOTEUR_DROIT = STOP + MARCHE_AR;
            *COMMANDE_MOTEUR_GAUCHE = STOP;
        }
        else if (TEST_JOY_BAS(vTestJoy) && TEST_JOY_DROITE(vTestJoy)) {
            *COMMANDE_MOTEUR_DROIT = STOP;
            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;
        }
        else if (TEST_JOY_HAUT(vTestJoy)) {
            *COMMANDE_MOTEUR_DROIT = STOP + MARCHE_AV;
            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
        }
        else if (TEST_JOY_BAS(vTestJoy)) {
            *COMMANDE_MOTEUR_DROIT = STOP + MARCHE_AR;
            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;
        }
        else if (TEST_JOY_DROITE(vTestJoy)) {
            *COMMANDE_MOTEUR_DROIT = STOP + MARCHE_AV;
            *COMMANDE_MOTEUR_GAUCHE = STOP;
        }
        else if (TEST_JOY_GAUCHE(vTestJoy)) {
            *COMMANDE_MOTEUR_DROIT = STOP;
            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
        }
        else {
            *COMMANDE_MOTEUR_DROIT = STOP;
            *COMMANDE_MOTEUR_GAUCHE = STOP;
        }

```

Programme avec masque

6) Périphérique de commande des moteurs

Le périphérique utilise 4 registres de 16 bits accessibles en écriture seulement.

le robot possède 4 roues motrices indépendantes deux à deux. Pour accéder aux 4 roues, on utilise les adresses : 0x20, 0x22, 0x24 et 0x26.

On réalise ensuite un programme principal élémentaire qui permet de configurer les moteurs et de les mettre en rotation en fonction de la direction joystick sélectionnée :

```
while (1) {
    short j = (*CAPTEURS_JOYSTICK) & 0x001F;

    if (j & 0x0001) { // bouton -> stop prioritaire
        *COMMANDE_MOTEUR_DROIT  = STOP;
        *COMMANDE_MOTEUR_GAUCHE = STOP;
    }
    else if (j == 16) { // haut
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV;
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
    }
    else if (j == 8) { // bas
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR;
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;
    }
    else if (j == 2) { // droite
        *COMMANDE_MOTEUR_DROIT  = STOP;
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
    }
    else if (j == 4) { // gauche
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV;
        *COMMANDE_MOTEUR_GAUCHE = STOP;
    }
    else if (j == 18) { // haut+droite
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV/2;
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
    }
    else if (j == 20) { // haut+gauche
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV;
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV/2;
    }
    else if (j == 10) { // bas+droite
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR/4;
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;
    }
    else if (j == 12) { // bas+gauche
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR;
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR/4;
    }
    else {
        *COMMANDE_MOTEUR_DROIT  = STOP;
        *COMMANDE_MOTEUR_GAUCHE = STOP;
    }
}
return 0;
```

Programme de commande des moteurs

7) Périphérique de mesure distance

Ce périphérique permet de mesurer la distance d'un objet situé à proximité du robot. Le robot possède 3 périphériques identiques pour détecter la présence à droite, au centre et à gauche d'un obstacle.

Un périphérique comporte 4 registres internes :

— **3 accessibles en écriture seulement :**
  — `REGISTRE_IMPULSION` : 5000
  — `REGISTRE_PERIODE` : 1
  — `REGISTRE_LIMITE` : Distance limite de détection en cm

— **1 registre en lecture seulement :**
  — `REGISTRE_DISTANCE`

Zone 5		Adresse	A_5	A_4	A_3	A_2	A_1	A_0	
$\overline{C_{S5}}$ 4*16bits	Base+40	0x28	1	0	1	0	0	0	REGISTRE_IMPULSION
	Base+41	0x29	1	0	1	0	0	1	
	Base+42	0x2A	1	0	1	0	1	0	REGISTRE_PERIODE
	Base+43	0x2B	1	0	1	0	1	1	
	Base+44	0x2C	1	0	1	1	0	0	REGISTRE_LIMITE
	Base+45	0x2D	1	0	1	1	0	1	
	Base+46	0x2E	1	0	1	1	1	0	REGISTRE_DISTANCE
	Base+47	0x2F	1	0	1	1	1	1	

Exemple pour le capteur central

On réalise un programme principal élémentaire qui permet de configurer le capteur de distance central et d'afficher la distance mesurée sur l'afficheur 2. Puis on fait de même pour les capteurs droite et gauche.

```
18  void main() {
19
20      short capteurs;
21      short vTestJoy;
22      short vdistCentre, vdistDroite, vdistGauche;
23
24      initialiser_capteurs();
25      *COMMANDE_MOTEUR_MAITRE = 1000;
26
27      *COMMANDE_MOTEUR_DROIT = STOP;
28      *COMMANDE_MOTEUR_GAUCHE = STOP;
29
30      while(1) {
31
32          capteurs = *CAPTEURS_ROBOT;
33          vTestJoy = *CAPTEURS_JOYSTICK;
34
35          if ( TEST_CAPTEUR_CENTRE(capteurs) == 0 ) {
36              *COMMANDE_MOTEUR_GAUCHE = STOP + (MARCHE_AV / 2);
37              *COMMANDE_MOTEUR_DROIT  = STOP + (MARCHE_AV / 2);
38          }
39
40          else if ( TEST_CAPTEUR_GAUCHE(capteurs) == 0 ) {
41              *COMMANDE_MOTEUR_GAUCHE = STOP;
42              *COMMANDE_MOTEUR_DROIT  = STOP + (MARCHE_AV / 2);
43          }
44
45          else if ( TEST_CAPTEUR_DROIT(capteurs) == 0 ) {
46              *COMMANDE_MOTEUR_GAUCHE = STOP + (MARCHE_AV / 2);
47              *COMMANDE_MOTEUR_DROIT  = STOP;
48          }
49
50          vdistCentre = *ECHO_CENTRE;
51          vdistDroite = *ECHO_DROITE;
52          vdistGauche = *ECHO_GAUCHE;
53
54          PILOTAGE_AFFICHEUR_1(vdistCentre);
55          PILOTAGE_AFFICHEUR_2(vdistDroite);
56          PILOTAGE_AFFICHEUR_3(vdistGauche);
57      }
58  }
```

Programme des capteurs de distance

8) Périphérique de gestion du capteur de ligne

Ce périphérique utilise 3 capteurs de ligne au sol qui renvoient 0 s' ils voient une ligne noire au sol et renvoient 1 sinon. On écrit 3 sous-programmes permettant de renvoyer un 1 logique lorsqu'un capteur voit la ligne noire et 0 sinon.

```
vtestLigne = *CAPTEURS_ROBOT;
PILOTAGE_AFFICHEUR_2(vtestLigne);
```

Programme du capteur de ligne

IV) Présentation des différents algorithmes de contrôle

Dans cette partie, nous étudions comment le robot se déplace selon le cahier des charges.

1) Mode 0 : Joy v0

Pour le mode joy v0, le robot se déplace simplement en suivant les directions indiquées par le joystick. Si l'utilisateur relâche le joystick, le robot s'arrête immédiatement. De plus, une sécurité est intégrée : si les capteurs à ultrasons détectent un obstacle à moins de 30 cm, le robot se bloque et seule la marche arrière reste autorisée pour s'éloigner du danger.
On réalise le programme suivant :

```
void MODE_JOY_V0(void)
{
    short vtestJoy;
    unsigned short vdistGauche, vdistCentre, vdistDroite;

    *COMMANDE_MOTEUR_MAITRE = 1000;
    *COMMANDE_MOTEUR_DROIT  = STOP;
    *COMMANDE_MOTEUR_GAUCHE = STOP;
    *START_MOTEURS          = 7;

    *COMMANDE_TRIGGER_ASTABLE_CENTRE  = 5000;
    *COMMANDE_TRIGGER_RAPPORT_CENTRE  = 1;
    *ECHO_LIMITE_CENTRE               = 5000;

    *COMMANDE_TRIGGER_ASTABLE_DROITE  = 5000;
    *COMMANDE_TRIGGER_RAPPORT_DROITE  = 1;
    *ECHO_LIMITE_DROITE               = 5000;

    *COMMANDE_TRIGGER_ASTABLE_GAUCHE  = 5000;
    *COMMANDE_TRIGGER_RAPPORT_GAUCHE  = 1;
    *ECHO_LIMITE_GAUCHE               = 5000;

    while (1) {
        vdistCentre = *ECHO_CENTRE;
        vdistGauche = *ECHO_GAUCHE;
        vdistDroite = *ECHO_DROIT;
        vtestJoy    = *CAPTEURS_JOYSTICK;

        PILOTAGE_AFFICHEUR_2(vdistCentre);
        PILOTAGE_AFFICHEUR_1(vdistGauche);
        PILOTAGE_AFFICHEUR_3(vdistDroite);
        /* distances + affichage */
vdistCentre = *ECHO_CENTRE;
vdistGauche = *ECHO_GAUCHE;
vdistDroite = *ECHO_DROIT;
vtestJoy    = *CAPTEURS_JOYSTICK;

PILOTAGE_AFFICHEUR_2(vdistCentre);
PILOTAGE_AFFICHEUR_1(vdistGauche);
PILOTAGE_AFFICHEUR_3(vdistDroite);

/* distance minimale*/
unsigned short minDist = vdistCentre;
if (vdistGauche < minDist) minDist = vdistGauche;
if (vdistDroite < minDist) minDist = vdistDroite;
```

```

if (minDist < SEUIL_OBSTACLE) {
    if (TEST_JOY_BAS(vtestJoy)) {
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR;
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;
    } else {
        *COMMANDE_MOTEUR_DROIT  = STOP;
        *COMMANDE_MOTEUR_GAUCHE = STOP;
    }
} else {
    if (vtestJoy == JOY_STOP) {
        PILOTAGE_AFFICHEUR_1(9);

    } else if (vtestJoy == JOY_HAUT_GAUCHE) {
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV/2;
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV;

    } else if (vtestJoy == JOY_HAUT_DROITE) {
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV/2;
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;

    } else if (vtestJoy == JOY_BAS_GAUCHE) {
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR/4;
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR;

    } else if (vtestJoy == JOY_BAS_DROITE) {
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR/4;
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;

    } else if (vtestJoy == JOY_DROIT) {
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;

    } else if (vtestJoy == JOY_GAUCHE) {
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV;

    } else if (vtestJoy == JOY_BAS) {
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR;
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;

    } else if (vtestJoy == JOY_HAUT) {
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV;
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;

    } else if (vtestJoy==3 || vtestJoy==5 || vtestJoy==9 || vtestJoy==11 ||
               vtestJoy==13 || vtestJoy==17 || vtestJoy==19 || vtestJoy==21) {
        *COMMANDE_MOTEUR_DROIT  = STOP;
        *COMMANDE_MOTEUR_GAUCHE = STOP;

    } else {
        *COMMANDE_MOTEUR_DROIT  = STOP;
        *COMMANDE_MOTEUR_GAUCHE = STOP;
    }
}

```

2) Mode 1 : Joy v1

Ce mode de déplacement fonctionne de la même manière que le mode JOY V0 : le robot suit les directions données par l'utilisateur via le joystick. Cependant, deux règles importantes ont été ajoutées :

— **Priorité à l'arrêt :** la commande d'arrêt est désormais prioritaire. Si l'utilisateur demande une direction tout en appuyant sur le bouton d'arrêt, le robot choisira systématiquement de s'immobiliser.

— **Mémorisation du mouvement :** une fois une direction est lancée, le robot continue d'avancer dans cette voie même si le joystick est relâché. Il faut soit appuyer sur le bouton d'arrêt, soit donner une nouvelle commande pour stopper ou changer la trajectoire.

Le programme réalisé est le suivant :

```
void JOY_V1(void)
{
    short vtestJoy;
    unsigned short vdistCentre, vdistGauche, vdistDroite;

    while (1) {
        vdistCentre = *ECHO_CENTRE;
vdistGauche = *ECHO_GAUCHE;
vdistDroite = *ECHO_DROIT;
vtestJoy    = *CAPTEURS_JOYSTICK;

PILOTAGE_AFFICHEUR_2(vdistCentre);
PILOTAGE_AFFICHEUR_1(vdistGauche);
PILOTAGE_AFFICHEUR_3(vdistDroite);

unsigned short minDist = vdistCentre;
if (vdistGauche < minDist) minDist = vdistGauche;
if (vdistDroite < minDist) minDist = vdistDroite;

if (minDist < SEUIL_OBSTACLE) {
    if (TEST_JOY_BAS(vtestJoy)) {
        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR;
        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;
    } else {
        *COMMANDE_MOTEUR_DROIT  = STOP;
        *COMMANDE_MOTEUR_GAUCHE = STOP;
    }
} else {

    if (vtestJoy == JOY_HAUT) lastDirection = vtestJoy;

    if (vtestJoy == 0 || vtestJoy != lastDirection) {

        if (lastDirection == JOY_STOP) {
            PILOTAGE_AFFICHEUR_1(9);

        } else if (lastDirection == JOY_HAUT_GAUCHE) {
            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV/2;
            *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV;

        } else if (lastDirection == JOY_HAUT_DROITE) {
            *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV/2;
            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
```

```
        } else if (lastDirection == JOY_BAS_GAUCHE) {
            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR/4;
            *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR;

        } else if (lastDirection == JOY_BAS_DROITE) {
            *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR/4;
            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;

        } else if (lastDirection == JOY_HAUT) {
            /* rien */

        } else if (lastDirection == JOY_DROIT) {
            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;

        } else if (lastDirection == JOY_GAUCHE) {
            *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV;

        } else if (lastDirection == JOY_BAS) {
            *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR;
            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;

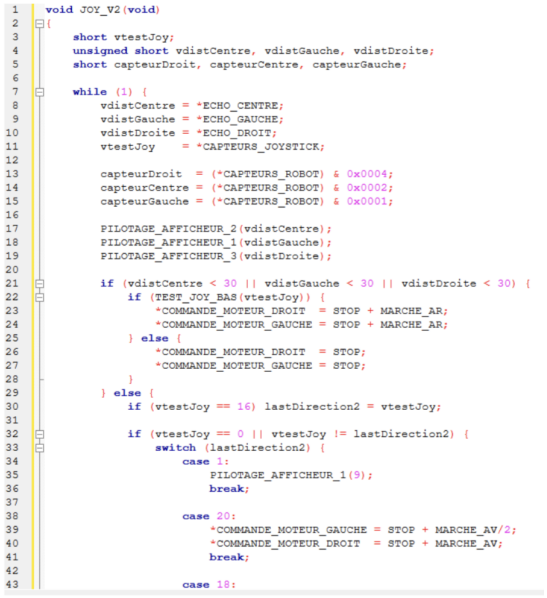
        } else if (lastDirection==3 || lastDirection==5 || lastDirection==9 || lastDirection==11 ||
                   lastDirection==13 || lastDirection==17 || lastDirection==19 || lastDirection==21) {
            *COMMANDE_MOTEUR_DROIT  = STOP;
            *COMMANDE_MOTEUR_GAUCHE = STOP;

        } else {
            *COMMANDE_MOTEUR_DROIT  = STOP;
            *COMMANDE_MOTEUR_GAUCHE = STOP;
        }
    }
}
```

3) Mode 2 : Autonome

Nous étudions maintenant un mode de déplacement autonome où le robot, d'abord en attente sur une ligne noire, s'active quand l'interrupteur sw[9] est sur 1 pour suivre la ligne au sol. s'il croise une ligne perpendiculaire et que ses trois capteurs détectent du noir, il continue d'avancer tout en ajoutant un point à un compteur de barres affiché sur l'afficheur 2 (si sw0 est à 0). Enfin, si un obstacle est détecté à moins de 30 cm, le robot s'arrête et seule la marche arrière reste possible pour se dégager.

Le programme réalisé est le suivant :



```
void JOY_V2(void)
{
    short vtestJoy;
    unsigned short vdistCentre, vdistGauche, vdistDroite;
    short capteurDroit, capteurCentre, capteurGauche;

    while (1) {
        vdistCentre = *ECHO_CENTRE;
        vdistGauche = *ECHO_GAUCHE;
        vdistDroite = *ECHO_DROIT;
        vtestJoy    = *CAPTEURS_JOYSTICK;

        capteurDroit  = (*CAPTEURS_ROBOT) & 0x0004;
        capteurCentre = (*CAPTEURS_ROBOT) & 0x0002;
        capteurGauche = (*CAPTEURS_ROBOT) & 0x0001;

        PILOTAGE_AFFICHEUR_2(vdistCentre);
        PILOTAGE_AFFICHEUR_1(vdistGauche);
        PILOTAGE_AFFICHEUR_3(vdistDroite);

        if (vdistCentre < 30 || vdistGauche < 30 || vdistDroite < 30) {
            if (TEST_JOY_BAS(vtestJoy)) {
                *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR;
                *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;
            } else {
                *COMMANDE_MOTEUR_DROIT  = STOP;
                *COMMANDE_MOTEUR_GAUCHE = STOP;
            }
        } else {
            if (vtestJoy == 16) lastDirection2 = vtestJoy;

            if (vtestJoy == 0 || vtestJoy != lastDirection2) {
                switch (lastDirection2) {
                    case 1:
                        PILOTAGE_AFFICHEUR_1(9);
                        break;

                    case 20:
                        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV/2;
                        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV;
                        break;

                    case 18:
                        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV/2;
                        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
                        break;

                    case 12:
                        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR/4;
                        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR;
                        break;

                    case 10:
                        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR/4;
                        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;
                        break;

                    case 2:
                        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
                        break;

                    case 4:
                        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV;
                        break;

                    case 8:
                        *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AR;
                        *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AR;
                        break;

                    case 16:
                        break;

                    case 3: case 5: case 9: case 11: case 13: case 17: case 19: case 21:
                        *COMMANDE_MOTEUR_DROIT  = STOP;
                        *COMMANDE_MOTEUR_GAUCHE = STOP;
                        break;

                    default:
                        if (capteurCentre || (capteurGauche && capteurDroit)) {
                            BAR_COMPT++;
                            *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV;
                            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
                        } else if (capteurGauche) {
                            *COMMANDE MOTEUR DROIT  = STOP + MARCHE AV;
                            *COMMANDE_MOTEUR_GAUCHE = STOP;
                        } else if (capteurDroit) {
                            *COMMANDE_MOTEUR_DROIT  = STOP;
                            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
                        } else {
                            *COMMANDE_MOTEUR_DROIT  = STOP;
                            *COMMANDE_MOTEUR_GAUCHE = STOP;
                        }
                        break;
                }
            }
        }
    }
}
```

4) Mode 3 : Esclave

Enfin, nous modifions le programme précédent pour que le robot fonctionne en mode "esclave". Une fois placé derrière un obstacle à moins de 50 cm et activé par l'interrupteur sw[9], le robot avance en gardant toujours cette distance de sécurité avec lui. S'il perd le contact avec l'obstacle, il s'arrête net et affiche la lettre a sur tous ses écrans. Dès que l'obstacle réapparaît, le robot reprend automatiquement sa route et recommence à le suivre.
Le programme réalisé est le suivant :

```
void JOY_V3(void)
{
    short vtestJoy;
    unsigned short vdistCentre, vdistGauche, vdistDroite;

    while (1) {
        vdistCentre = *ECHO_CENTRE;
        vdistGauche = *ECHO_GAUCHE;
        vdistDroite = *ECHO_DROIT;
        vtestJoy    = *CAPTEURS_JOYSTICK;

        if (vdistCentre > DIST_OBSTACLE && vdistCentre < (DIST_OBSTACLE + DIST_MIN_OBSTACLE)) {
            *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV;
            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
        } else if (vdistDroite > DIST_OBSTACLE) {
            *COMMANDE_MOTEUR_DROIT  = STOP + MARCHE_AV;
            *COMMANDE_MOTEUR_GAUCHE = STOP;
        } else if (vdistGauche > DIST_OBSTACLE) {
            *COMMANDE_MOTEUR_DROIT  = STOP;
            *COMMANDE_MOTEUR_GAUCHE = STOP + MARCHE_AV;
        } else {
            *AFFICHEUR_1 = 10;
            *AFFICHEUR_2 = 10;
            *AFFICHEUR_3 = 10;
            *COMMANDE_MOTEUR_DROIT  = STOP;
            *COMMANDE_MOTEUR_GAUCHE = STOP;
        }
    }
}
```

V) Conclusion

Ce bureau d'études nous a permis de mettre en pratique les notions d'algorithmique, de programmation et d'architecture des ordinateurs. Dans la partie matérielle, nous avons configuré le processeur du robot en étudiant le décodage d'adresses et les échanges entre les composants. Nous avons aussi testé les périphériques en langage c avec le logiciel nios. Même si la notion de mapping a été difficile à comprendre au début, nous avons fini par réussir nos tests. Ensuite, pour la partie logicielle, nous avons programmé et validé plusieurs stratégies de déplacement. malgré quelques difficultés techniques qui nous ont ralentis, il a été très intéressant de voir le robot exécuter nos algorithmes en temps réel. Enfin, nous remercions nos encadrants pour leur aide précieuse tout au long de ce projet.